

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
По лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом

Студент гр. 4345

Беденков А.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

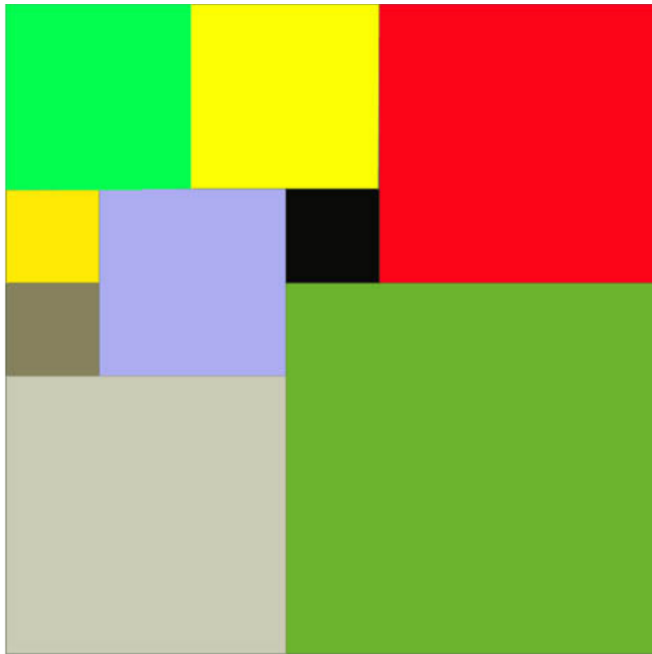
Цель работы.

Изучить и реализовать алгоритм поиска с возвратом.

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу – квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков (квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы – одно целое число N ($2 \leq N \leq 20$).

Выходные данные

Одно число K , задающее минимальное количество обрезков (квадратов), из которых можно построить столешницу (квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых числа x , y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка (квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Основные теоретические положения.

Поиск с возвратом — это универсальная стратегия для решения задач, требующих полного перебора возможных комбинаций в заданном пространстве вариантов.

Суть подхода заключается в постепенном построении решения шаг за шагом. На каждом этапе алгоритм пытается добавить следующий элемент к текущему частичному решению. Если обнаруживается, что продолжение по этому пути невозможно или ведёт к тупику, метод откатывается назад и выбирает альтернативный вариант. Это позволяет исследовать все потенциальные ветви дерева решений.

Ключевое преимущество метода — возможность находить все допустимые решения задачи, если таковые существуют. Для повышения эффективности часто применяются эвристики и критерии отсечения: неперспективные варианты отсекаются как можно раньше, ещё до их полного раскрытия. Благодаря этому удаётся существенно сократить объём вычислений и ускорить получение результата.

Выполнение работы.

Вариант 2и.

Итеративный бэктрекинг. Исследование времени выполнения от размера квадрата.

Постановка задачи и стратегия решения.

Рассматривается задача покрытия квадратной области размерами $N \times N$ минимальным числом квадратных плиток с целочисленными сторонами, не превышающими N . Для повышения эффективности алгоритма решение разбито на три логических блока, учитывающих математические свойства числа N :

- **Случай четного N .**

Если сторона столешницы четная, задача решается аналитически за постоянное время. Оптимальное разбиение всегда состоит из ровно четырех квадратов размером $N/2 \times N/2$, расположенных по четвертям исходного квадрата.

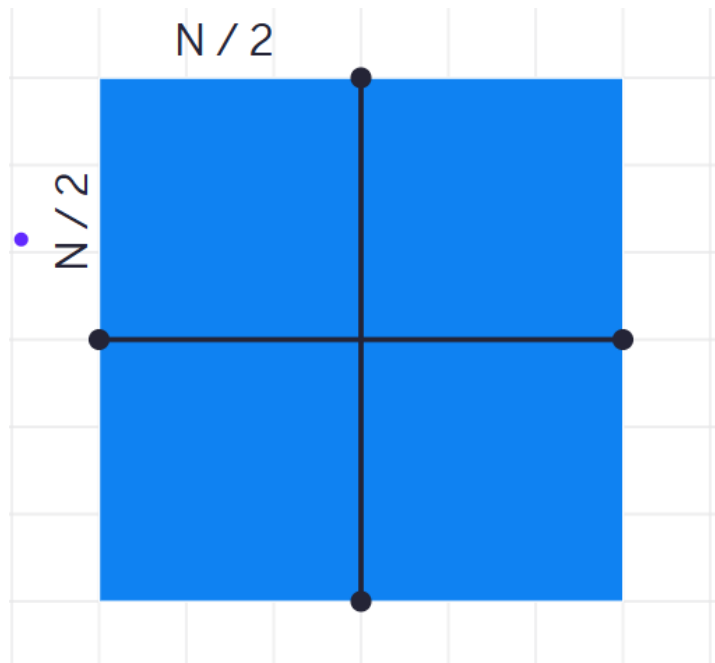


Рисунок 1 – Заполнение столешницы для чётных N

- **Случай составного нечетного N .**

При нечетном значении N , являющемся составным числом, применяется метод масштабирования. Алгоритм находит наименьший простой делитель p числа N . Задача сводится к оптимизации для квадрата меньшего размера $p \times p$. Полученное оптимальное разбиение затем масштабируется множителем $m = N/p$. Это позволяет значительно сократить пространство поиска, избегая перебора лишних вариантов для больших чисел.

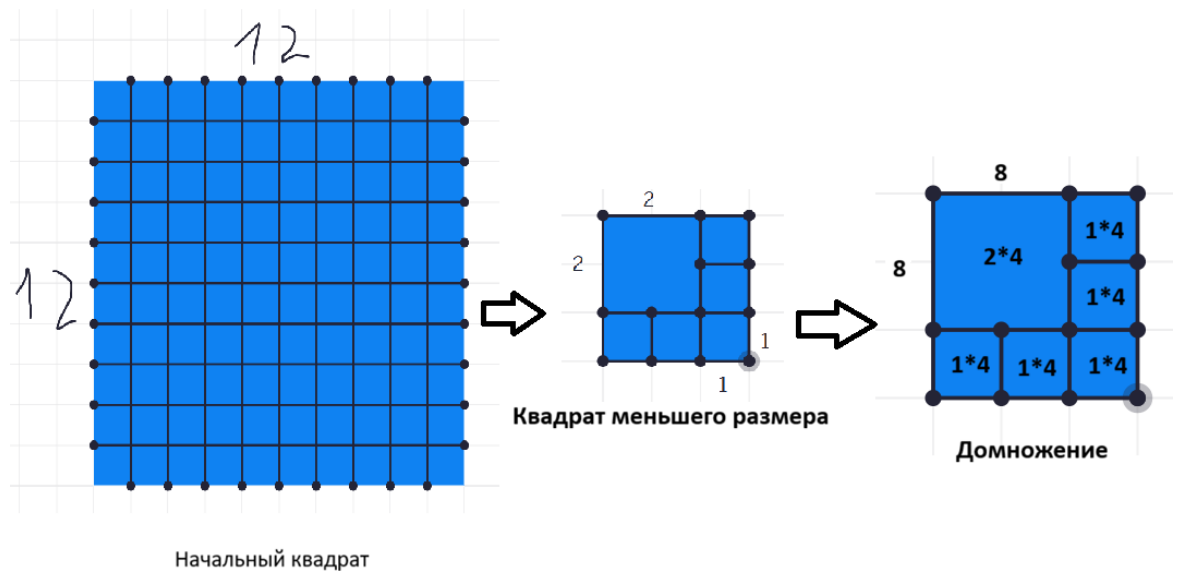


Рисунок 2 – Заполнение столешницы для нечётных составных N

- **Случай простого нечетного N .**

Это наиболее сложный случай, требующий применения полного перебора с эвристикой. В качестве начального состояния используется доказательство о том, что для оптимального разбиения часто выгодным является размещение трех крупных квадратов по углам.

- В левый верхний угол ставится квадрат со стороной $\lfloor N/2 \rfloor + 1$.
- Справа и снизу от него размещаются два квадрата со стороной $\lfloor N/2 \rfloor$. Оставшаяся свободная область заполняется методом итеративного бэктрекинга.

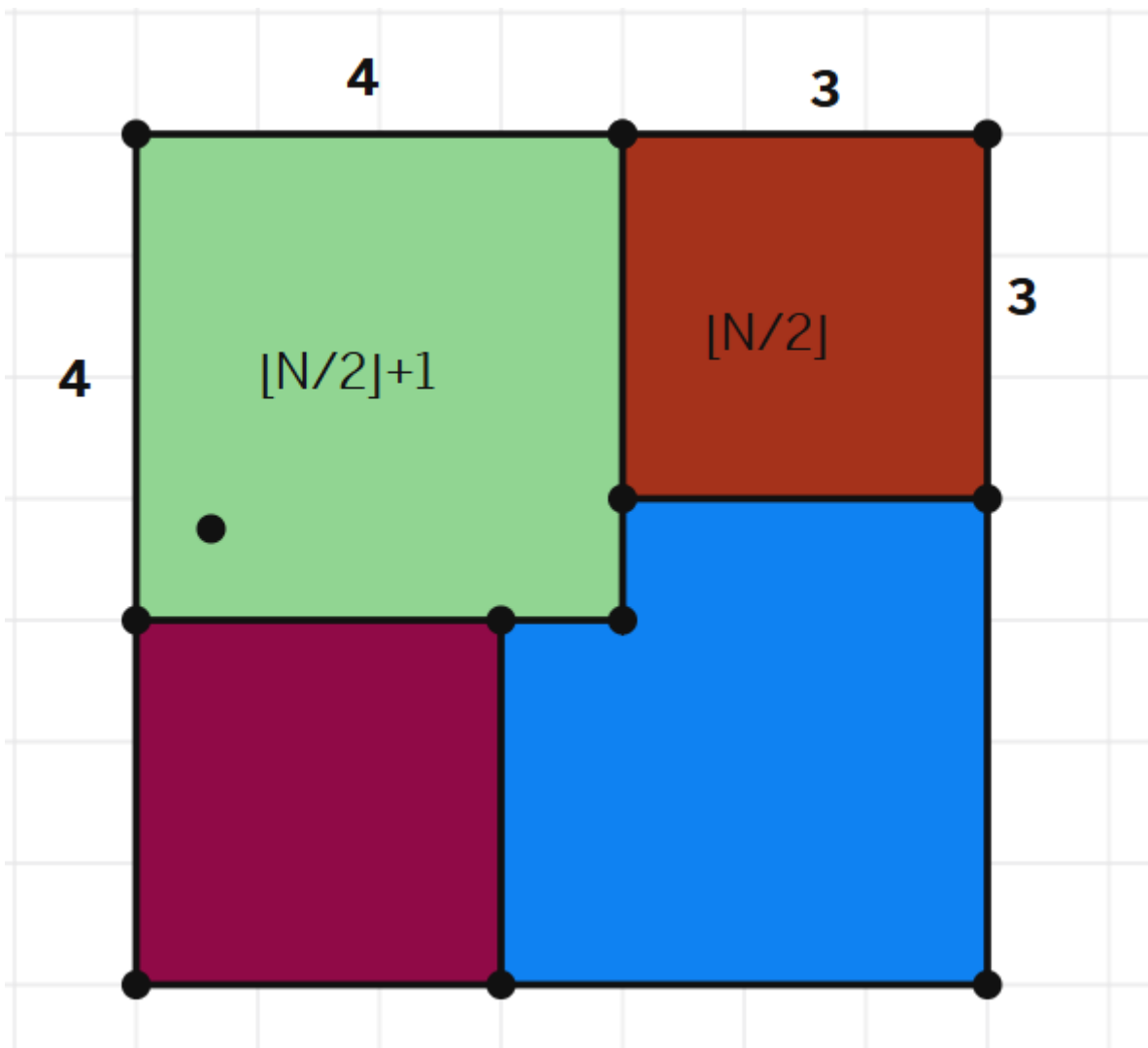


Рисунок 3 – Заполнение столешницы для нечётных простых N

Алгоритм заполнения.

Для управления состоянием поля используется одномерный массив профиля высот (cols), где каждый элемент хранит текущую занятость соответствующего вертикального столбца. Это снижает потребление памяти и ускоряет проверку доступности ячеек.

Поиск решений строится на стеке вызовов, что исключает риск переполнения рекурсивного стека при работе с большими значениями N . Цикл алгоритма работает по следующему принципу:

1. Поиск самой нижней свободной точки (минимум в массиве высот).
2. Расчет максимально возможного размера квадрата, который может поместиться в эту позицию без выхода за границы поля и наложения на уже поставленные фигуры.

3. Добавление выбранного квадрата в текущее решение и сохранение контекста в стек.
4. Проверка условий завершения или необходимости отката (если текущее количество фигур превышает найденный минимум или поле заполнено).

При откате последний добавленный квадрат удаляется из поля и стека, его размер уменьшается на единицу, и поиск продолжается с новой конфигурации. Если варианты для позиции исчерпаны, алгоритм возвращается к предыдущему уровню вложенности через стек.

Реализация включает следующие функциональные блоки:

- 1) `solve_square_iterative(N)` — основная функция-оркестратор.

Является точкой входа в алгоритм. Принимает размер столешницы N , определяет математические свойства числа и в зависимости от них выбирает стратегию решения: мгновенный возврат ответа для чётных чисел, масштабирование для составных или полный перебор для простых.

- 2) Внутренняя логика масштабирования.

Вычисляет коэффициент $m = N / p$ сразу после нахождения наименьшего делителя p . После завершения перебора каждый квадрат из найденного решения преобразуется к исходному масштабу: координаты умножаются на m и сдвигаются на $+1$ для перевода в систему счёта с единицы.

- 3) Управление стеком и профилем высот.

Реализует нерекурсивный поиск с возвратом через два взаимосвязанных объекта: одномерный массив ``cols``, хранящий текущую заполненную высоту каждого столбца, и стек ``stack``, содержащий контекст каждого размещённого квадрата. При добавлении квадрата высоты в ``cols`` увеличиваются, при откате — уменьшаются обратно. Это позволяет избежать полного пересчёта состояния поля на каждом шаге.

4) `solve_and_time(N)` — вспомогательная функция замера производительности.

Является внешней обёрткой над основным алгоритмом. Фиксирует момент времени до и после вызова `solve_square_iterative` с помощью `time.perf_counter()`, после чего форматировано выводит количество квадратов, их координаты и размеры, а также итоговое время выполнения в секундах.

Оценка сложности алгоритма

Сложность алгоритма по времени: $O(c^n)$

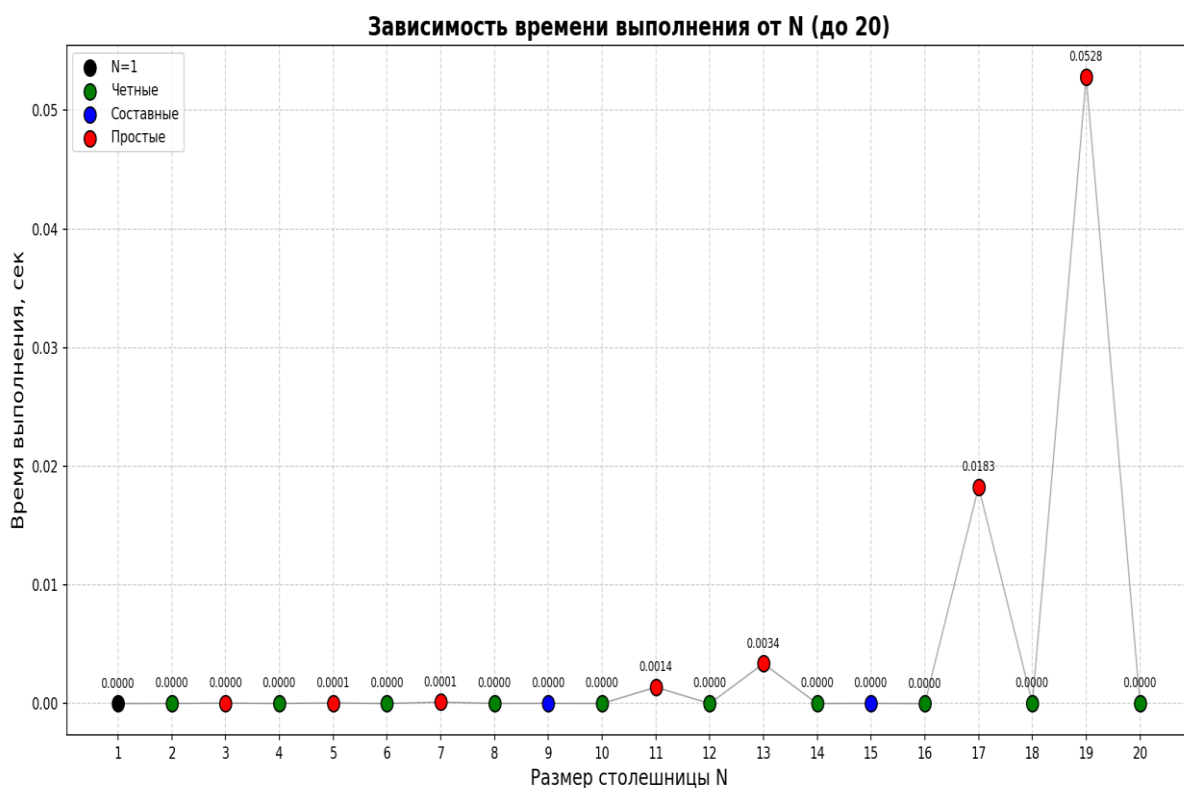
Сложность алгоритма по памяти: $O(n^2)$

Таблица 1 – Тестирование программы

Размер столешницы	Результат	Комментарий
20	4 1 1 10 11 1 10 1 11 10 11 11 10	Размер столешницы – чётное число
15	6 1 1 10 11 1 5 1 11 5 11 6 5 6 11 5 11 11 5	Размер столешницы – нечётное составное число
13	11 1 1 7 8 1 6 1 8 6 8 7 2 10 7 4	Размер столешницы – нечётное простое число

	7 8 1	
	7 9 3	
	10 11 1	
	11 11 3	
	7 12 2	
	9 12 2	

Было проведено исследование зависимости времени выполнения программы от размера столешницы N . Результаты исследования см. на Рис. 4



Анализ графика (Рис. 4):

Четные N выполняются мгновенно ($O(1)$, аналитическое решение из 4 квадратов).

Нечетные составные N — чуть медленнее (примерно 0.0001 сек) за счет масштабирования задачи к малому делителю.

Нечетные простые N — пики времени, т.к. требуется полный перебор (экспонента).

Выводы

В ходе выполнения лабораторной работы был разработан и реализован алгоритм минимального разбиения квадратной области со стороной N на квадраты меньшего размера методом поиска с возвратом.

Для повышения производительности в алгоритм были внедрены дополнительные оптимизации: определение чётности входного числа и нахождение его наименьшего простого делителя. Это позволило значительно сократить время работы программы для чётных и нечётных составных чисел, сведя задачу к решению для квадрата меньшего размера с последующим масштабированием результата. Реализация выполнена итеративно с использованием явного стека вместо рекурсии, что исключает риск переполнения стека вызовов.

Проведённое исследование зависимости времени выполнения от размера столешницы подтвердило экспоненциальный характер роста времени для нечётных простых чисел. Для обеспечения приемлемой скорости работы был реализован механизм отсечения бесперспективных ветвей: ветки перебора, в которых количество фигур уже превышает текущий найденный минимум, немедленно отбрасываются без дальнейшего исследования.

Корректность работы программы была подтверждена тестированием на наборе входных данных, включающем все три типа чисел. Во всех случаях программа успешно завершила работу и вывела результат в требуемом формате.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import time

def solve_square_iterative(N):
    if N == 1:
        print(f"N = 1, тривиальный случай — один квадрат 1x1")
        return [(1, 1, 1)]

    p = N
    for i in range(2, int(N**0.5) + 1):
        if N % i == 0:
            p = i
            break
    m = N // p

    if p != N:
        print(f"Найден наименьший делитель p = {p}, применяем масштаб x{m}")
        print(f"Задача сведена к квадрату {p}x{p}")
    else:
        print(f"N = {N} является простым числом, масштабирование невозможно")
        print(f"Запускаем полный перебор для квадрата {p}x{p}")

    if p == 2:
        print(f"Чётный размер столешницы — делим на 4 квадрата размером {m}x{m}")
        squares = [(0, 0, 1), (1, 0, 1), (0, 1, 1), (1, 1, 1)]
        return [(x * m + 1, y * m + 1, w * m) for x, y, w in squares]

    half = p // 2
    first_sq = half + 1

    print(f"\nРазмещаем три начальных квадрата:")
    print(f"  Квадрат {first_sq}x{first_sq} в левый верхний угол (0, 0)")
    print(f"  Квадрат {half}x{half} справа от первого ({first_sq}, 0)")
    print(f"  Квадрат {half}x{half} снизу от первого (0, {first_sq})")

    cols = [p] * half + [first_sq] + [half] * half
    best_sol = []
    min_count = [p * p]
    current_sol = [
        (0, 0, first_sq),
        (first_sq, 0, half),
        (0, first_sq, half)
    ]
    stack = []

    iteration = 0

    while True:
        min_y = min(cols)
        min_x = cols.index(min_y)
```

```

need_backtrack = False

if min_y == p:
    print(f"\n Все клетки заняты!")
    if len(current_sol) < min_count[0]:
        min_count[0] = len(current_sol)
        best_sol = list(current_sol)
        print(f" Новый рекорд: {min_count[0]} квадратов")
        need_backtrack = True

    elif len(current_sol) >= min_count[0]:
        print(f" Количество квадратов ({len(current_sol)}) >=
текущего минимума ({min_count[0]}) – отсечение")
        need_backtrack = True

if need_backtrack:
    while stack:
        last_x, last_y, last_w, last_max_w = stack.pop()

        for i in range(last_x, last_x + last_w):
            cols[i] -= last_w
        current_sol.pop()

        new_w = last_w - 1
        if new_w > 0:
            print(f" Откат: уменьшаем квадрат в позиции
({last_x}, {last_y}) с {last_w} до {new_w}")
            for i in range(last_x, last_x + new_w):
                cols[i] += new_w
            current_sol.append((last_x, last_y, new_w))
            stack.append([last_x, last_y, new_w, last_max_w])
            break
        else:
            print(f" Квадрат в позиции ({last_x}, {last_y})
исчерпан – извлекаем следующий из стека")
            else:
                print(f"\nСтек пуст – перебор завершён")
                break
            continue

    max_w = 1
    for i in range(min_x + 1, p):
        if cols[i] == min_y:
            max_w += 1
        else:
            break
    max_w = min(max_w, p - min_y)

    iteration += 1
    print(f"\nИтерация {iteration}: свободная позиция ({min_x},
{min_y}), "
          f"размещаем квадрат {max_w}x{max_w}, "
          f"всего квадратов в решении: {len(current_sol) + 1}")

    for i in range(min_x, min_x + max_w):
        cols[i] += max_w
    current_sol.append((min_x, min_y, max_w))

```

```

        stack.append([min_x, min_y, max_w, max_w])

    print(f"\nЛучшее найденное решение: {min_count[0]} квадратов")
    print(f"Применяем масштаб x{m} и переводим координаты ")

    result = []
    for x, y, w in best_sol:
        rx, ry, rw = x * m + 1, y * m + 1, w * m
        print(f"  ({x}, {y}, {w}) -> ({rx}, {ry}, {rw})")
        result.append((rx, ry, rw))

    return result

def solve_and_time(N):
    print(f"=" * 50)
    print(f"Начало работы программы, N = {N}")
    print(f"=" * 50)

    start = time.perf_counter()
    result = solve_square_iterative(N)
    end = time.perf_counter()

    print(f"\n{'=' * 50}")
    print(f"Результат работы программы:")
    print(f"{'=' * 50}")
    print(len(result))
    for x, y, size in result:
        print(f"{x} {y} {size}")
    print(f"\nВремя выполнения: {end - start:.6f} секунд")

if __name__ == "__main__":
    solve_and_time(int(input()))

```